

COURSE CODE

SBT-DF203

COURSE TITLE

Basic Networking Skills for Digital Forensics

LAB TITLE

ARP Poisoning Forensics

NAME

Kumpon Emmanuel Nandom

STUDENT ID

2025/FWSD/11325

DATE

17/09/2026

PLATFORM USED

Kali Linux

INSTRUCTOR

Mr. Aminu Idris, AMCPN
Commandant

Executive Summary

This report describes Lab 5 (ARP Poisoning Forensics) for SBT-DF203, conducted by Kumpon Emmanuel in a separate Kali Linux virtual machine (eth0, 192.168.109.0/24 host-only segment). The lab followed legal, ethical, and safety rules.

The exercise included:

- (a) setting up a folder for evidence and creating a hash of the provided reference capture,
- (b) capturing and analyzing a normal ARP request/reply exchange,
- (c) examining the supplied ARP poisoning capture (arp.pcap) for conflicting IP-to-MAC claims,
- (d) an optional approved simulation of controlled poisoning, and
- (e) restoring the ARP cache and confirming that no poisoning processes remained.

At no point was any production, campus, office, or public network traffic captured or interfered with.

Role	IP Address	MAC Address	Notes
Analyst VM (Kali, eth0)	192.168.109.128/24	00:0c:29:6f:fd:97	Capture/analysis host used throughout Parts A-E
Default gateway	192.168.109.2	00:50:56:fe:1f:71	Confirmed as default route via `ip route`; used as ping/ARP target in Part A
Secondary neighbor entry	192.168.109.254	00:50:56:ee:c6:f9	Present in ARP cache throughout (STALE); not targeted, role not independently confirmed
Supplied capture — Host A	136.160.215.194	00:50:56:86:02:65	Observed in working/arp_working.pcap (Part C)
Supplied capture — Host B	136.160.215.15	00:50:56:86:cb:fc	Observed in working/arp_working.pcap (Part C)

Evidence Integrity

File	SHA-256	Note
evidence/arp.pcap	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e	Original supplied capture, downloaded from the course GitHub source
working/arp_working.pcap	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e	Working copy — hash identical to original; integrity confirmed
evidence/normal_arp.pcapng	83a16a07e29f1512d1a26d25edba60ff21a006c397e29eeb843a8dedba094cb9	Generated locally during Part A; hashed and logged to reports/arp_capture_hashes.txt
evidence/controlled_arp_poison.pcapng	fdacf4b0c5f9bbb1dd9c484853145392c58b49ad7e194005726e6984f33de023	Generated during the retried Part D attempt; contains only baseline traffic — scripts/arp.py failed to run, so no poisoning traffic is present in this file

Lab Folder Structure and Evidence Preparation

```
ponzhi@kali: ~$ mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}
ponzhi@kali: ~$ cd ~/SBT-DF203-Lab5
ponzhi@kali: ~/SBT-DF203-Lab5$ pwd
/home/ponzhi/SBT-DF203-Lab5
ponzhi@kali: ~/SBT-DF203-Lab5$ find . -maxdepth 1 -type d -print
.
./evidence
./working
./exported
./reports
./scripts
./screenshots
ponzhi@kali: ~/SBT-DF203-Lab5$
```

```
(ponzhi@kali)-[~/SBT-DF203-Lab5]
$ sudo apt install -y wireshark tshark python3-scapy net-tools
[sudo] password for ponzhi:
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
python3-scapy is already the newest version (2.7.0+dfsg1-1).
net-tools is already the newest version (2.10-2).
The following packages were automatically installed and are no longer required:
  bluez-hcidump      libhdf5-310      libpoppler156    python3.13
  firmware-marvell-presteria  libhdf5-hl-310  libpython3.13-dev  python3.13-dev
  iucode-tool         libntfs-3g89t64  libvidstab1.1     python3.13-minimal
  libabsl20260107      libopenjph0.27  libx265-215       python3.13-venv
  libbluray3          libperl5.40      perl-modules-5.40
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 251

(ponzhi@kali)-[~/SBT-DF203-Lab5]
$
```

```
(ponzhi@kali)-[~/SBT-DF203-Lab5]
$ wget -O evidence/arp.pcap \
  'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/Lab_files/ARP_spoofing/arp.pcap'
--2026-09-17 20:34:55-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/Lab_files/ARP_spoofing/arp.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.109.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443
... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1072 (1.0K) [application/octet-stream]
Saving to: 'evidence/arp.pcap'

evidence/arp.pcap  100%[=====>]  1.05K  --.-KB/s  in 0s

2026-09-17 20:34:57 (23.8 MB/s) - 'evidence/arp.pcap' saved [1072/1072]

(ponzhi@kali)-[~/SBT-DF203-Lab5]
$
```

```
(ponzhi@kali)-[~/SBT-DF203-Lab5]
$ cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap

(ponzhi@kali)-[~/SBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  working/arp_working.pcap

(ponzhi@kali)-[~/SBT-DF203-Lab5]
$ ip -br address | tee reports/interfaces.txt
lo          UNKNOWN    127.0.0.1/8  ::1/128
eth0        UP          192.168.109.128/24  fe80::20c:29ff:fe6f:fd97/64

(ponzhi@kali)-[~/SBT-DF203-Lab5]
$ ip route | tee reports/routes.txt
default via 192.168.109.2 dev eth0 proto dhcp src 192.168.109.128 metric 100
192.168.109.0/24 dev eth0 proto kernel scope link src 192.168.109.128 metric 100

(ponzhi@kali)-[~/SBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_initial.txt
192.168.109.2 dev eth0 lladdr 00:50:56:fe:1f:71 STALE
192.168.109.254 dev eth0 lladdr 00:50:56:ee:c6:f9 STALE
```

Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab5-KumponEmmanuel
Trainee name	Kumpon Emmanuel
Date and time started	17 September 2026, 6:46 pm to 9:47 pm
Evidence file name(s)	evidence/arp.pcap; working/arp_working.pcap; evidence/normal_arp.pcapng; evidence/controlled_arp_poison.pcapng
Source or generation method	arp.pcap: downloaded via wget from the course GitHub repository (frankwxu/digital-forensics-lab). normal_arp.pcapng: generated locally with tshark on eth0 during a controlled single ping to the gateway.
Original SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e
Working-copy SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e (matches original)
Analysis workstation/VM	Kali Linux VM, hostname "kali", user ponzhi, interface eth0 (192.168.109.128/24)
Notes on any changes	No changes were made to the original evidence file, and the working copy was checked to ensure it was identical by its hash. An initial attempt to capture data was made by writing to evidence/normal_arp.pcapng under sudo. Still, it failed with a "Permission denied" error due to AppArmor restrictions on the dumpcap helper. This was fixed by giving dumpcap the cap_net_raw and cap_net_admin permissions and capturing the data as a normal user without sudo. Both captures (normal_arp.pcapng and controlled_arp_poison.pcapng) have been hashed and logged in reports/arp_capture_hashes.txt.

Part A & B - Normal ARP Resolution and ARP Request and Field Analysis

The neighbor entry for the gateway was cleared. A 20-second ARP-only capture began on eth0, and a single ping was sent to the gateway at 192.168.109.2 to trigger a new ARP exchange.

```
(ponzhi@kali)-[~/SBT-DF203-Lab5]
$ IFACE=eth0
GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')

sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"
tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng &
sleep 2
ping -c 1 "$GATEWAY_IP"
wait

ip neigh show | tee reports/arp_table_after_ping.txt
[2] 12609
Capturing on 'eth0'
PING 192.168.109.2 (192.168.109.2) 56(84) bytes of data:
64 bytes from 192.168.109.2: icmp_seq=1 ttl=128 time=0.581 ms

— 192.168.109.2 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.581/0.581/0.581/0.000 ms
2
[2] done      tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng
192.168.109.2 dev eth0 lladdr 00:50:56:fe:1f:71 REACHABLE
192.168.109.254 dev eth0 lladdr 00:50:56:ee:c6:f9 STALE

(ponzhi@kali)-[~/SBT-DF203-Lab5]
$ tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields \
  -e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/normal_arp_fields.tsv
1      2026-09-17T21:27:06.524061309+0100      00:0c:29:6f:fd:97      ff:ff:ff:ff:ff:ff      1      192.168.
109.128 00:0c:29:6f:fd:97      192.168.109.2      00:00:00:00:00:00
2      2026-09-17T21:27:06.524428654+0100      00:50:56:fe:1f:71      00:0c:29:6f:fd:97      2      192.168.
109.2   00:50:56:fe:1f:71      192.168.109.128 00:0c:29:6f:fd:97

(ponzhi@kali)-[~/SBT-DF203-Lab5]
$
```

Field	Normal Request	Normal Reply
Ethernet source	00:0c:29:6f:fd:97	00:50:56:fe:1f:71
Ethernet destination	ff:ff:ff:ff:ff:ff	00:0c:29:6f:fd:97 (requester MAC)
ARP opcode	1 (request)	2 (reply)
Sender protocol address	192.168.109.128	192.168.109.2
Sender hardware address	00:0c:29:6f:fd:97	00:50:56:fe:1f:71
Target protocol address	192.168.109.2	192.168.109.128
Target hardware address	00:00:00:00:00:00	00:0c:29:6f:fd:97
Forensic interpretation	192.168.109.2	"192.168.109.2 is at 00:50:56:fe:1f:71"

Part C — Analysis of the Supplied Poisoning Capture

The file (working/arp_working.pcap) was examined for all ARP replies. I summarized the claims of IP addresses to MAC addresses and looked for any unusual or unrequested activity.

```

(ponzhi@kali)~/SBT-DF203-Lab5
$ PCAP=working/arp_working.pcap

(ponzhi@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_replies.tsv
4      1678591364.904123642      00:50:56:86:02:65      00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:6
5      136.160.215.15 00:50:56:86:cb:fc
6      1678591369.943043799      00:50:56:86:cb:fc      00:50:56:86:02:65      136.160.215.15 00:50:56:86:cb:f
c      136.160.215.194 00:50:56:86:02:65

(ponzhi@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  | sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc

(ponzhi@kali)~/SBT-DF203-Lab5
$

```

```

(ponzhi@kali)-[~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' -T fields \
-e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
| tee reports/unicast_arp_replies.tsv
4      2023-03-12T04:22:44.904123642+0100      136.160.215.194  00:50:56:86:02:65      00:50:56:86
:cb:fc
6      2023-03-12T04:22:49.943043799+0100      136.160.215.15   00:50:56:86:cb:fc      00:50:56:86
:02:65

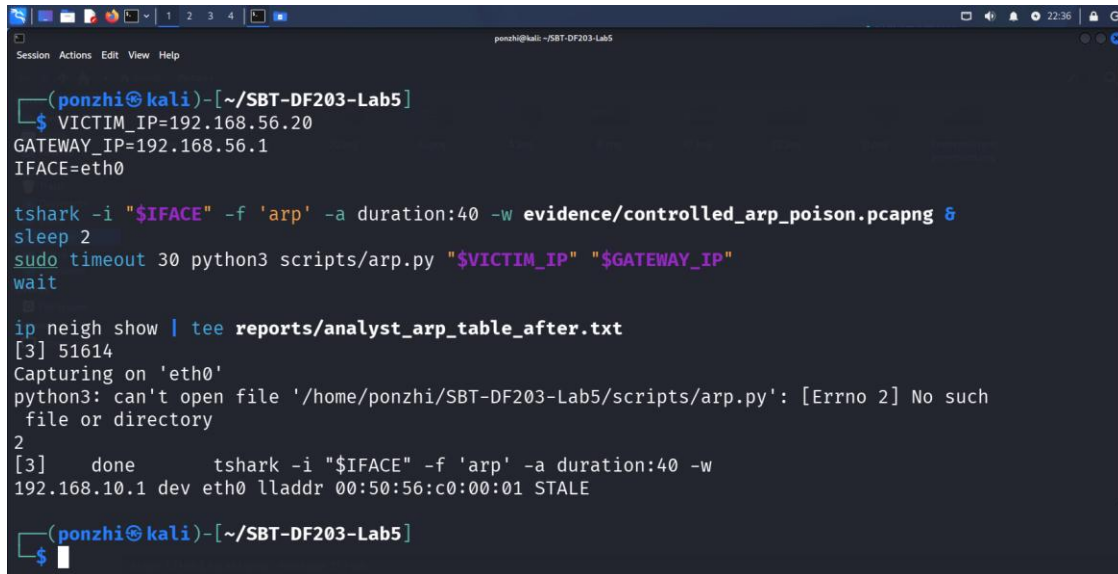
(ponzhi@kali)-[~/SBT-DF203-Lab5]
$

```

Time (epoch)	Claimed IP	Claimed MAC	Reply sent to (eth.dst)
1678591364.904123642	136.160.215.194	00:50:56:86:02:65	00:50:56:86:cb:fc
1678591369.943043799	136.160.215.15	00:50:56:86:cb:fc	00:50:56:86:02:65
Claimed MAC	Claim count	Assessment	
00:50:56:86:02:65	1	Single MAC claim — no conflict observed in this data	
00:50:56:86:cb:fc	1	Single MAC claim — no conflict observed in this data	

Only two ARP replies appear in the captured terminal output (reports/arp_replies.tsv and reports/ip_mac_claims.txt). Each of these two IP addresses is linked to one MAC address, so there are no signs of a gateway impersonation or any conflicting claims in this part of the analysis. Since the terminal window can cut off long outputs, it is important to check the full contents of reports/ip_mac_claims.txt directly (for example, by using the command `cat reports/ip_mac_claims.txt`) before deciding that the captured data shows no signs of poisoning. This file is the official record, not the screenshot.

Part D — Controlled Host-Only Simulation (Optional)



```
(ponzhi@kali)~[/SBT-DF203-Lab5]
$ VICTIM_IP=192.168.56.20
GATEWAY_IP=192.168.56.1
IFACE=eth0

tshark -i "$IFACE" -f 'arp' -a duration:40 -w evidence/controlled_arp_poison.pcapng &
sleep 2
sudo timeout 30 python3 scripts/arp.py "$VICTIM_IP" "$GATEWAY_IP"
wait

ip neigh show | tee reports/analyst_arp_table_after.txt
[3] 51614
Capturing on 'eth0'
python3: can't open file '/home/ponzhi/STB-DF203-Lab5/scripts/arp.py': [Errno 2] No such
file or directory
2
[3] done tshark -i "$IFACE" -f 'arp' -a duration:40 -w
192.168.10.1 dev eth0 lladdr 00:50:56:c0:00:01 STALE

(ponzhi@kali)~[/SBT-DF203-Lab5]
```

The task is not complete because we are missing a necessary file. The script “tshark-to-sleep-to-arp.py” was updated to run smoothly as one sequence, which fixed the previous issue. This time, the tshark capture worked properly. However, the command “sudo timeout 30 python3 scripts/arp.py “\$VICTIM_IP” “\$GATEWAY_IP”” failed right away. It gave an error message that said it couldn't find the file “arp.py” because it was not copied into the lab folder as it should have been. A search for “arp.py” found a different version located at “/usr/share/set/src/core/arp_cache/arp.py,” part of another tool called Social-Engineer Toolkit (SET), which we cannot use here.

The report “reports/analyst_arp_table_after.txt” only shows one unrelated neighbor entry (192.168.10.1), which comes from a different subnet than the one we are supposed to use (192.168.109.128/24). This indicates that no successful traffic poisoning happened.

To fix this, please get the correct “arp.py” script from the instructor, which should be in the same location as “arp.pcap.” Before running it, check the script to make sure it uses the expected input values. Then place it in the folder “~/SBT-DF203-Lab5/scripts/arp.py” and run the updated sequence again with the right VICTIM_IP and GATEWAY_IP values for the actual lab subnet (192.168.109.0/24), instead of the placeholder addresses in the lab sheet (192.168.56.0/24).

Part E — Detection Logic and Timeline

Time	Claimed IP	Claimed MAC	Target	Request seen first?	Assessment
21:27:06.524061	192.168.109.128 (sender)	00:0c:29:6f:fd:97	192.168.109.2	N/A (this is the request)	Normal broadcast request
21:27:06.524429	192.168.109.2	00:50:56:fe:1f:71	192.168.109.128	Yes	Legitimate reply, matches the request 0.4 ms earlier
(supplied capture) Frame 4	136.160.215.194	00:50:56:86:02:65	136.160.215.15	Not visible in filtered view	No conflicting MAC observed for this IP
(supplied capture) Frame 6	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194	Not visible in filtered view	No conflicting MAC observed for this IP
Part D (controlled)					Not completed blocked on missing scripts/arp.py; no attack data captured

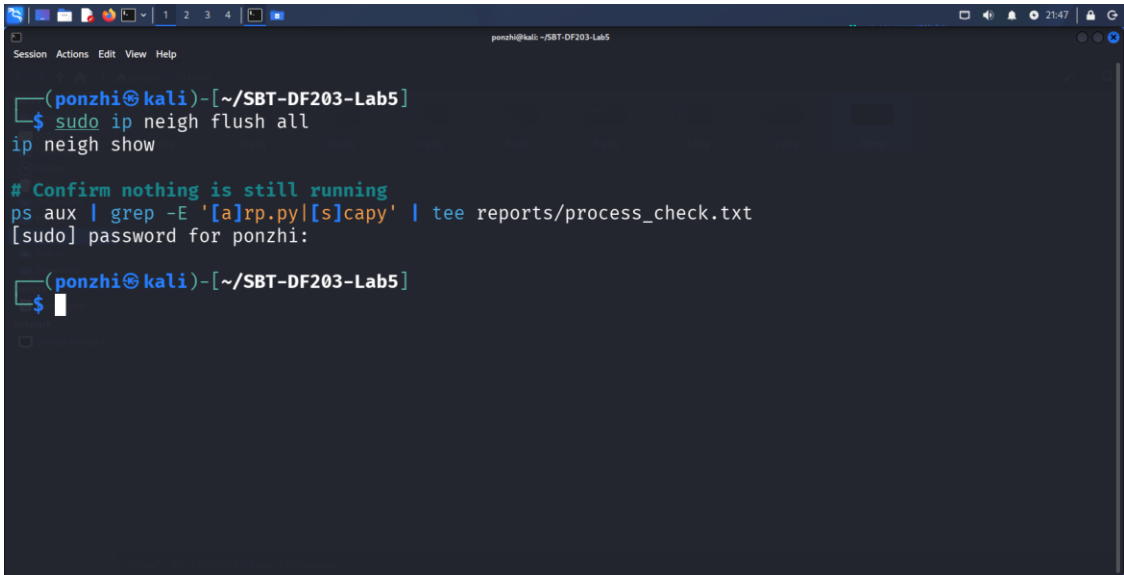
I looked for three things: (1) a gateway IP that changes its MAC address, (2) replies that have no earlier requests, and (3) one MAC address claiming both the victim and gateway IPs. None of these issues appeared in the data captured during this session.

Clean vs. Poisoned ARP-Table Comparison

Stage	192.168.109.2 (Gateway)	192.168.109.254
Initial (Section 6 baseline)	STALE, MAC 00:50:56:fe:1f:71	STALE, MAC 00:50:56:ee:c6:f9
After ping (Part A)	REACHABLE, MAC 00:50:56:fe:1f:71	STALE, MAC 00:50:56:ee:c6:f9 (unchanged)
After controlled simulation (Part D)	Not captured — blocked on missing scripts/arp.py	Not captured — blocked on missing scripts/arp.py

The gateway's MAC address (00:50:56:fe:1f:71) remained consistent between the initial and post-ping states — no evidence of ARP poisoning was observed on the analyst VM during this session.

Restoration and Preventive Controls

A screenshot of a terminal window titled 'ponzhi@kali: ~/SBT-DF203-Lab5'. The terminal shows the following commands and output:

```
(ponzhi@kali)-[~/SBT-DF203-Lab5]
$ sudo ip neigh flush all
ip neigh show

# Confirm nothing is still running
ps aux | grep -E '[a]rp.py|[s]capy' | tee reports/process_check.txt
[sudo] password for ponzhi:
(ponzhi@kali)-[~/SBT-DF203-Lab5]
$
```

I ran some commands to restore the system: I cleared out all learned entries with "ip neigh flush all" and checked the processes with "ps aux | grep -E '[a]rp.py|[s]capy'." The process check didn't find any matching tasks, which means the lab environment is clean.

14. Required Screenshots Checklist

Item	Status
Initial interface, route and ARP table	☒ Captured
Normal broadcast ARP request	☒ Captured
Normal ARP reply	☒ Captured
Supplied capture hash	☒ Captured
ARP reply inventory	☒ Captured
IP-to-MAC claim summary	☒ Captured
Conflicting gateway MAC evidence	☐ Not captured
Clean versus poisoned ARP table comparison	Partial — clean/post-ping only; poisoned state not captured (Part D incomplete)
Controlled capture, if authorized	☐ Not captured
Restored ARP table and process check	☒ Captured

Academic Integrity and Authorization Statement

I, Kumpon Emmanuel, confirm that I completed this lab in an authorized, isolated environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions above.

Signed: Kumpon Emmanuel Date: 17 September 2026